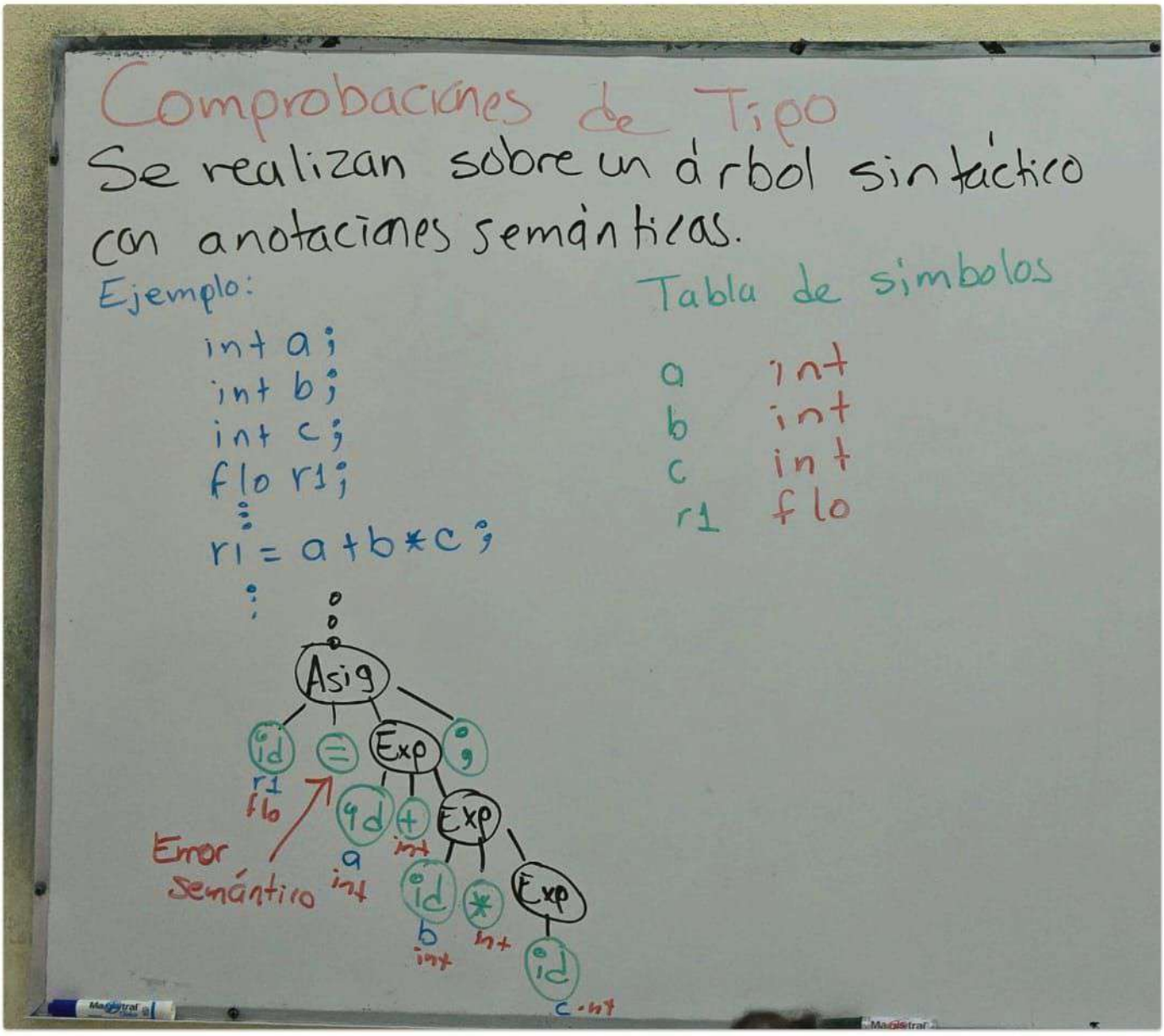
```
r1 = a + b * c;
```



Tema 2 - Generación de código intermedio

Para realizar esta traducción, el compilador hace uso de los árboles de derivación generados durante la fase de análisis sintáctico (árboles sintácticos), pero enfocándose principalmente en las porciones del árbol que representan las operaciones o instrucciones que verdaderamente intervienen en la construcción del programa ejecutable, a estas porciones de árbol se les identifica como árboles de expresiones y son ampliamente utilizadas por el generador de código intermedio para traducir el código a una notación diferente

Árbol de expresiones

Estos representan de una forma jerárquica natural el orden de ejecución de operaciones y ayuda a la traducción de las operaciones a una notación más cercana al propio lenguaje máquina.

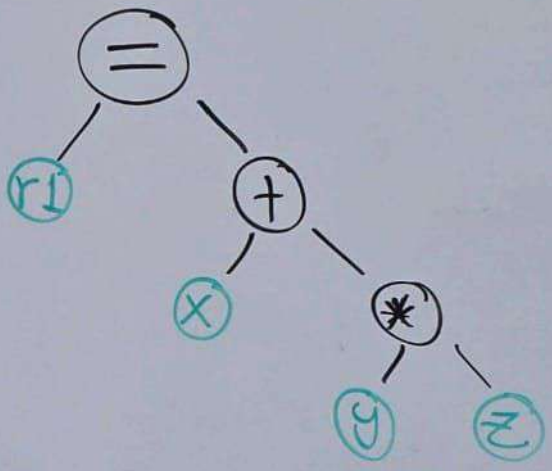
Por ejemplo

TEMA II. GENERACIÓN DE CÓDIGO INTERMEDIO

Árbol de Expresiones

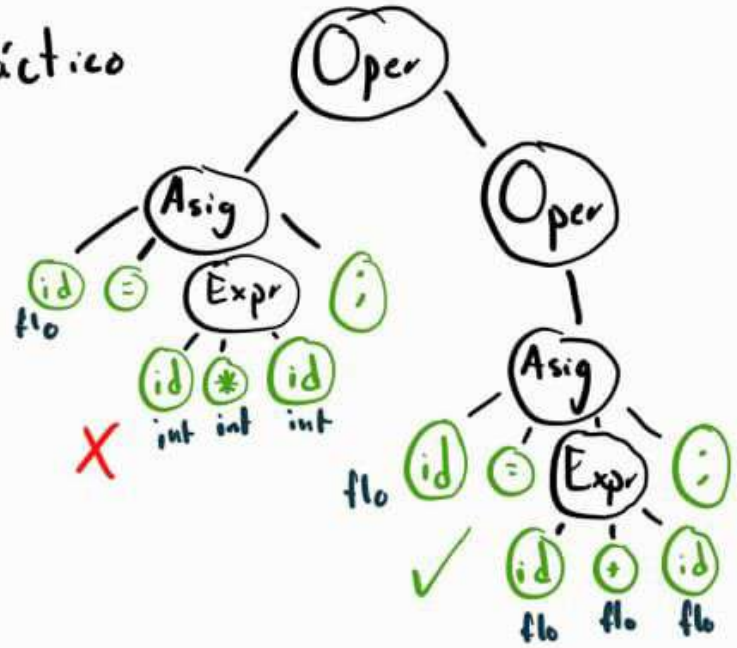
$r_1 = x + y * z$

(3) (2) (1)



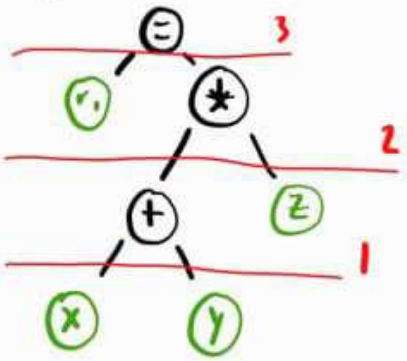
Árbol sintáctico

```
int x;
int y;
flo z;
flo r1;
flo r2;
  :
r1 = x * y;
r2 = r1 + z;
```

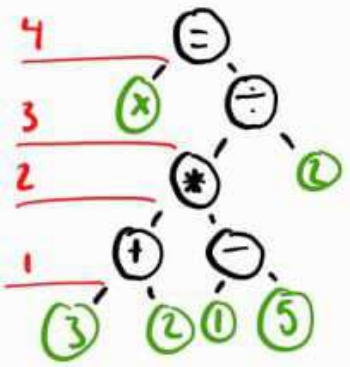


Árbol de expresiones

$r_1 = (x + y) * z$
3 1 2



$x = (3,2) * (5-1) \div 2$
5 1 3 2 4



a) $q = z + x * y - w \div 2$
5 3 1 4 2



b) $q = (z + x) * (y - w) \div 2$
5 1 3 2 4



c) $q = z + x * (y - w \div 2)$
4 3 2 1



Tipos de notaciones

Existen 3 tipos de notaciones para las expresiones de cualquier lenguaje, estas son:

- notación infija
- notación prefija
- notación posfija

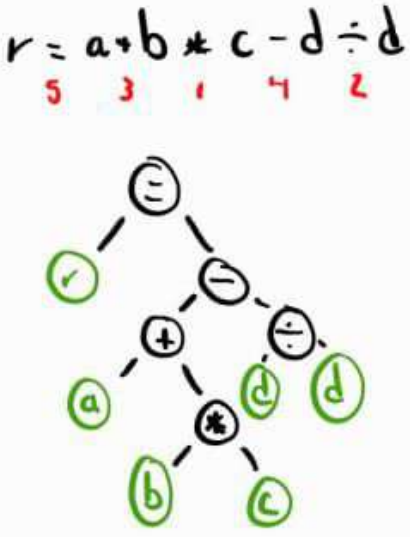
Cada una de estas notaciones tiene ciertas características, ventajas y desventajas respecto a las otras. **Todas** pueden obtenerse a partir del **árbol de expresiones,** por lo que este nos sirve como herramienta para traducir o transformar de una notación a otra.

Notación infija

Esta es la utilizada comúnmente en el lenguaje natural humano, en ella se encuentra ubicado en medio de sus operandos, por ejemplo:

r = a + b * c - d / d

Este tipo de notación se obtiene haciendo un recorrido en in-orden del árbol de la expresión

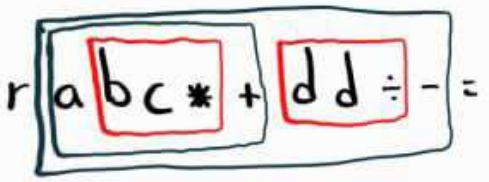


El recorrido en in-orden extrae la información de los nodos internos cuando se regresa a él desde la izquierda del árbol

Notación posfija

En este tipo de notación, el operador se ubica a la derecha de sus operandos, y se obtiene haciendo un recorrido en post-orden del árbol de la expresión.

Recordemos que en el recorrido en post-orden, la información de los nodos internos se extrae únicamente cuando se regresa al nodo desde la derecha del árbol, lo que implica que es la última vez que se accederá al nodo



Notación prefija

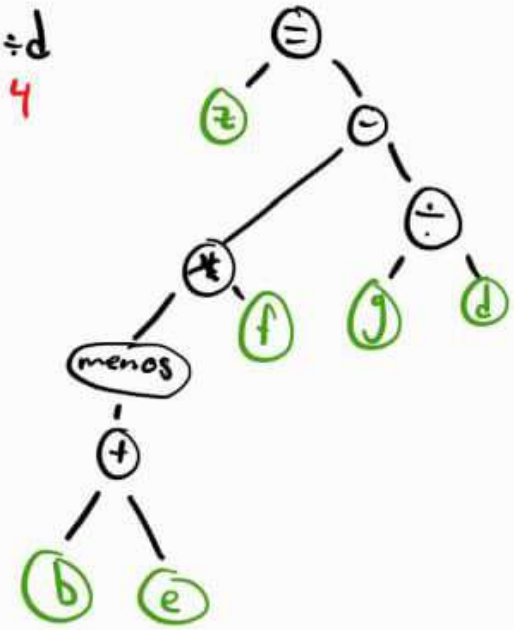
En este tipo de notación, el operador se ubica a la izquierda de sus operandos, y se obtiene haciendo un recorrido en pre-orden del árbol de la expresión

En el recorrido en pre orden la información de los nodos se extrae la primera vez que se visita

= r - + a * b c ÷ d d

Ejercicio $z = -(b+e)*f-g\div d$

6 2 1 3 5 4



Postfija

z b e + (menos) f * g d ÷ - =

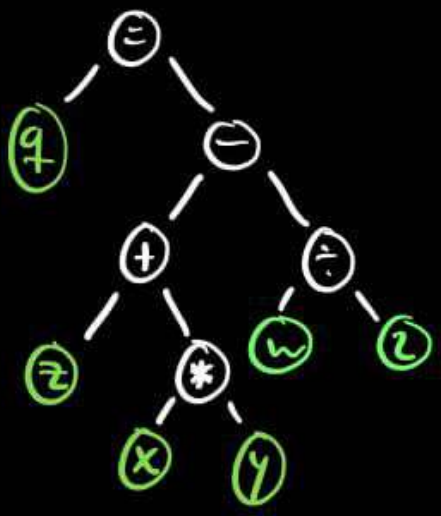
Prefija

= z - * (menos) + b e f ÷ g d

$q = z + x * y - w / 2$

Prefijo: $= q - + z * x y \div w 2$

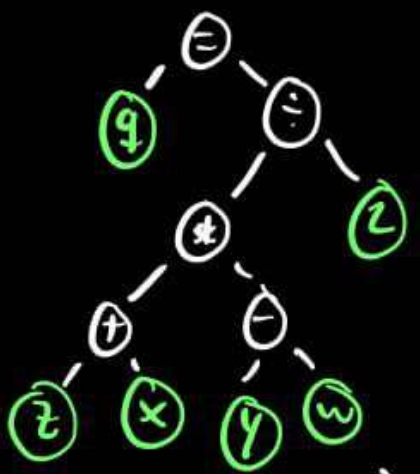
Postfija: $q z x y * + w 2 \div - =$



$q = (z + x) * (y - w) / 2$

Prefijo: $= q \div * + z x - y w 2$

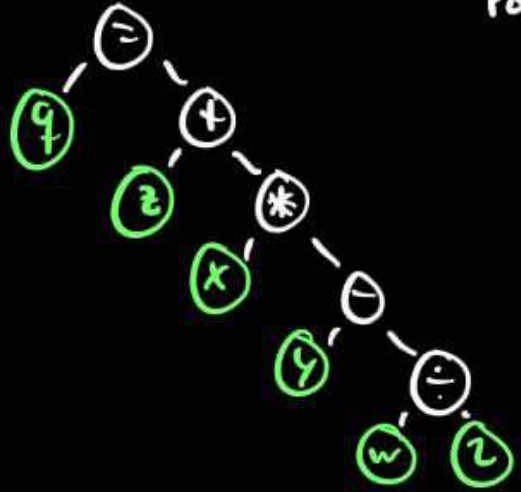
Postfija: $q z x + y w - * 2 \div =$



$q = z + x * (y - w / 2)$

Prefija: $= q + z * x - y \div w 2$

Postfija: $q z x y w 2 \div - * + =$



1) ¿Por qué se le llamó así?
2) ¿A cuál de las notaciones anteriores corresponde?
3) ¿Cuál es alguno de los beneficios de su uso en el área de las ciencias computacionales?
4) Menciona un lenguaje de programación que aplique este tipo de notación preferentemente que tú lo hayas utilizado alguna vez

---

Notación polaca

- ¿Por qué se le llamó así?
  Se le llamó Notación Polaca en honor a la nacionalidad de su creador, el lógico y matemático polaco Jan Łukasiewicz, quien la inventó alrededor de 1924 para simplificar la lógica proposicional eliminando la necesidad de usar paréntesis.

- ¿A cuál de las notaciones anteriores corresponde?
  Corresponde a la notación prefija, ya que en la notación polaca original el operador precede a los operandos (por ejemplo, + A B). (Nota: Su variante inversa, la Notación Polaca Inversa o RPN, corresponde a la postfija).

- ¿Cuál es alguno de los beneficios de su uso en el área de las ciencias computacionales?
  Elimina la necesidad de paréntesis y la ambigüedad en las reglas de precedencia de operadores. Esto simplifica drásticamente la fase de compilación e interpretación en lenguajes de programación, ya que las expresiones pueden ser evaluadas de manera lineal y eficiente utilizando estructuras de datos como pilas (stacks) o árboles sintácticos sin reinterpretar agrupaciones.

- Menciona un lenguaje de programación que aplique este tipo de notación, preferentemente que tú lo hayas utilizado alguna vez

  C# (o cualquier lenguaje donde implementes parsing de expresiones con estructuras de datos como Stack<T>). Aunque la sintaxis nativa de C# es infija, en el desarrollo de estructuras de datos y compiladores se utiliza internamente para evaluar expresiones aritméticas mediante algoritmos de ordenación por estaciones (Shunting-yard) y evaluación por pilas.
  Fuente de información para justificación

- Libro: Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). Compiladores: Principios, Técnicas y Herramientas (2.ª ed.). Pearson Educación.
- Capítulo / Tema: Capítulo 2 y 6 (Sintaxis, Traducción dirigida por la sintaxis y Evaluación de expresiones mediante notaciones intermedia/prefija/ postfija).

El lenguaje ensamblador utiliza la notación polaca pues se escribe primero la operación a realizar (mov, sub, add) y posteriormente se indican los datos de operación, muchas de las veces estos datos se encuentran almacenados en los registros del procesador

Código de tres direcciones

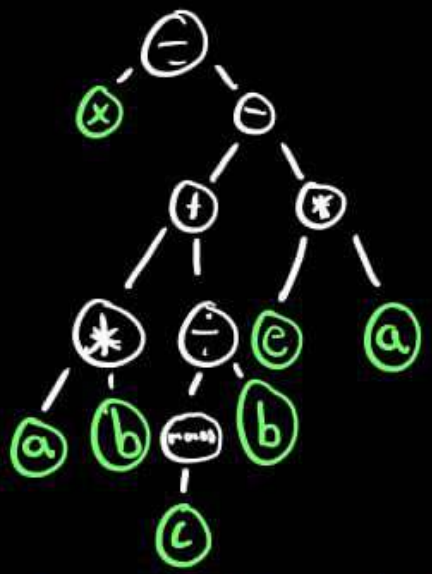
Durante la fase de generación de código intermedio, el programa fuente se traduce a una representación intermedia, esto es una forma de escritura mas abstracta que el lenguaje de alto nivel pero menos abstracta que el lenguaje máquina. Esta representación intermedia generalmente se expresa mediante un código de 3 direcciones en el que las instrucciones se escriben en una notación de la forma

$$id_1 := id_2 \ op \ id_3$$

id_1, id_2 y id_3 son identificadores (variables) y op es un operador

TAREA

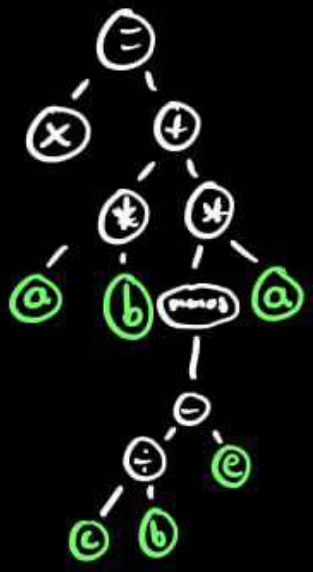
1) x = a * b + c ÷ b - e * a



Preorden = x - + * ab ÷ menos cb * ea

Postorden xab * c menos b ÷ + ea * - =

2) x = a * b + - (c ÷ b - e) * a



Preorden = x + * ab * menos - ÷ cbea

Postorden xab * cb ÷ e - menos a * + =

3) x = a * (b + - c) ÷ (b - e) * a



Preorden = x * ÷ * a + b menos c - bea

Postorden xabc menos + * ba - ÷ a * =

Preposiciones de código de tres direcciones

Existen diferentes tipos de preposiciones de tres direcciones, cada una de ellas con un propósito específico, pero todas ellas consideran a los identificadores como etiquetas de espacios definidos en la memoria. Entre los principales proposiciones para código de 3 direcciones se encuentran

1. Proposiciones de asignación de la forma

$$x = a \ _{op} \ b$$

2. Proposiciones de asignación de operaciones unarias

$$x = op\ y$$

3. Proposiciones de copia

$$x = y$$

4. Proposiciones para saltos condicionales

goto E

donde E corresponde a una etiqueta que marca el inicio de un bloque de código específico

5. Proposiciones para saltos condicionales

if x oprel y goto E

donde oprel es cualquier operador relacional como >, <, =, etc...

6. Proposiciones de llamada o procedimientos y de retorno desde procedimientos

```
param x1
param x2
  ⋮
param xn

call p,n
return y
```

7. Proposiciones de asignación en índices

x[i] = y ó x = y[i]

8. Proposiciones para trabajar con direcciones

x = &y
↓
Por referencia entre la dirección a la que apunta y

x = *y
↓
Por valor guarda en x el valor contenido en la celda que apunta y

*x = y
Por valor guarda en la celda apuntada por x el valor de y

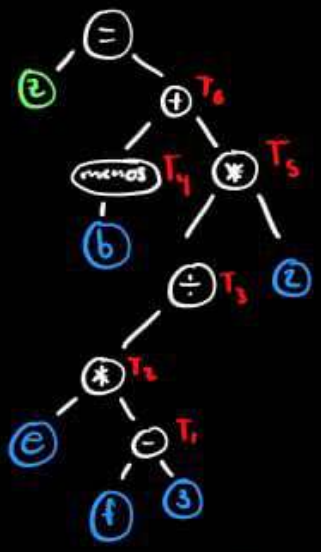
Traducción a código de tres direcciones

Para realizar la traducción a código de tres direcciones se siguen tres pasos de manera secuencial. Estos son:

1. Construir el árbol de la expresión
2. Identificar TODAS las variables temporales necesarias para contener los resultados parciales
3. Escribir la secuencia de expresiones en tres direcciones a partir del árbol

$z = -b + c * (f - 3) \div d * 2$

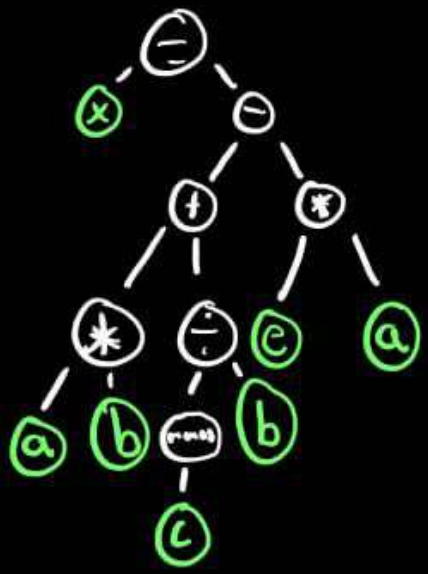
7 2 6 3 1 4 5



$T_1 = f - 3$
$T_2 = e * T_1$
$T_3 = T_2 \div d$
$T_4 = (menos)\, b$
$T_5 = T_3 * 2$
$T_6 = T_4 + T_5$
$z = T_6$

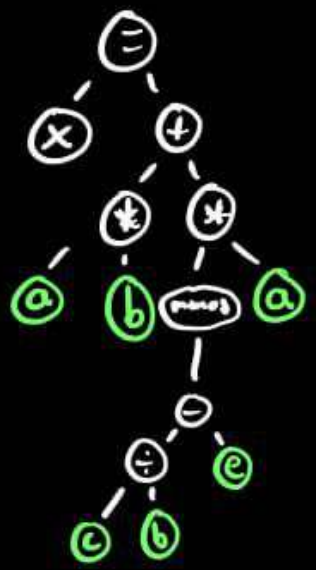
1) $x = a * b + - c \div b - e * a$

7 2 4 1 3 6 5



$T_1 = (menos)\, c$
$T_2 = a * b$
$T_3 = T_1 \div b$
$T_4 = T_2 + T_3$
$T_5 = e * a$
$T_6 = T_4 - T_5$
$x = T_6$

2) $x = a * b + -(c \div b - e) * a$



$T_1 = c \div b$

$T_2 = T_1 - e$

$T_3 = (menos)\, T_2$

$T_4 = a * b$

$T_5 = T_3 * a$

$T_6 = T_4 + T_5$

$x = T_6$

3) $x = a * (b + -c) \div (b - e) * a$



$T_1 = (menos)\, c$

$T_2 = b + T_1$

$T_3 = a * T_2$

$T_4 = b - e$

$T_5 = T_3 \div T_4$

$T_6 = T_5 * a$

$x = T_6$

Implementación del código de tres direcciones

La implementación del código de tres direcciones se realiza mediante la construcción de una tabla de registros con campos adecuados.
Ambas opciones son:

- Cuádruplos: incluye cuatro campos.
- Triples: incluye tres campos.

Esta forma de implementación facilita la traducción a lenguaje de bajo nivel como ensamblador.

Cuádruplos

Se compone de cuatro campos que incluyen los siguientes:

Operador, opd_izq, opd_der, resultado.

En esta tabla se registran de manera secuencial cada proposición de tres direcciones, incluyendo aquellas que implican variables temporales. Por ejemplo:

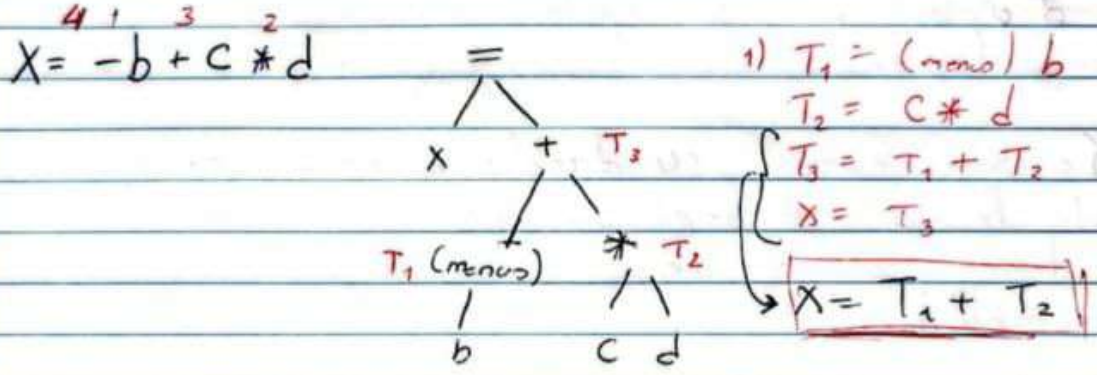
Cuádruplo

operador	opd_izq	opd_der	resultado
Menos		b	T_1
*	c	d	T_2
+	T_1	T_2	T_3
=		T_3	X
+	T_1	T_2	X

Expresión: x = -b + c * d

- Cuadruplo

operador (menos)	opd_izq	opd_der	resultado
*	c	d	T_1
+	T_1	T_2	T_2
=		T_3	T_3
			x
+	T_1	T_2	x



En las triples, la tabla tiene únicamente tres campos: operador, opd_izq, opd_der; es decir que la columna de resultado se elimina. Por lo tanto, en las operaciones que implican resultados parciales calculados, se utiliza el índice correspondiente dentro de la tabla en la posición en la cual se calcula el resultado parcial. Por ejemplo, el ejercicio anterior en la tabla de triples quedaría como:

i	operador	opd_izq	opd_der
1	-	a	b
2	-	y	z
3	*	(1)	5
4	÷	x	(2)
5	menos	———	z
6	-	(3)	(4)
7	+	(6)	(5)
8	=	w	(7)

c) Cuadruplo

	op	op_iz	op_dr	rco
$T_1 = X * Y$	*	X	Y	T_1
$T_2 = Z + T_1$	+	Z	T_1	T_2
$T_3 = W \div Z$	÷	W	Z	T_3
$T_4 = T_2 - T_3$	−	T_2	T_3	T_4
$r_2 = T_4$	=	——	T_4	r_2

Triplos

i	op	op iz	op dr
1	*	X	Y
2	+	Z	(1)
3	÷	W	Z
4	−	(2)	(3)
5	=	r_2	(4)

d)

$T_1 = Z + X$
$T_2 = Y - W$
$T_3 = T_1 * T_2$
$T_4 = T_3 \div Z$
$r = T_4$

op	op iz	op dr	rco
+	Z	X	T_1
−	Y	W	T_2
*	T_1	T_2	T_3
÷	T_3	Z	T_4
=	——	T_4	r

i	op	op iz	op dr
1	+	Z	X
2	−	Y	W
3	*	(1)	(2)
4	÷	(3)	Z
5	=	r	(4)

e)

$T_1 = w \div z$

$T_2 = y - T_1$

$T_3 = x * T_2$

$T_4 = z + T_3$

$r = T_4$

op	op 12	op dr	res
÷	w	z	T_1
−	y	T_1	T_2
*	x	T_2	T_3
+	z	T_3	T_4
=	——	T_4	r

i	op	op 12	op dr
1	÷	w	z
2	−	y	(1)
3	*	x	(2)
4	+	z	(3)
5	=	r	(4)

15 septiembre

Ejercicio en clase

18 septiembre

Ejercicio en clase

22 septiembre

ejercicio en clase

23 septiembre

Ejercicio en clase

29 septiembre

Tarea

2 de octubre

Apunte